Personal Finance Management System

Database Management System Project

Ngo Tuan Bach

Student ID: 11245846

Supervisor: Tran Hung

National Economics University

Faculty of Mathematical Economics

Presentation Outline

1. Project overview and system architecture
2. Database design and ERD structure
3. Normalization, constraints, and indexing
4. SQL implementation: views, procedures, functions, triggers
5. Security, testing, and evaluation
6. Backend/frontend summary and conclusion
7. Conclusion

Main Focus

This presentation focuses mainly on the MySQL database design and SQL implementation.

Project Overview

Main Purpose

The system helps users manage personal financial activities through a database-backed web application.

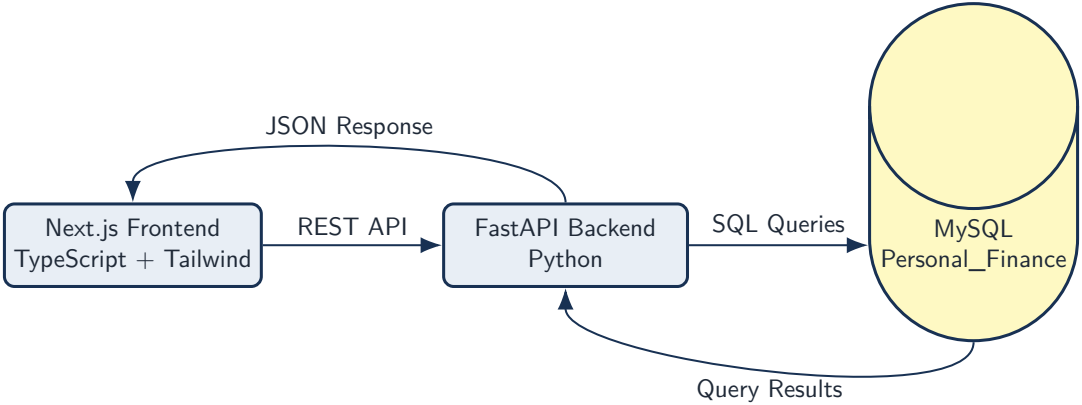
Main User Features

- User authentication
- Bank account tracking
- Income and expense management
- Budget planning
- Saving goal tracking

Database Features

- MySQL relational schema
- ERD-based design
- Views for reporting
- Procedures and functions
- Triggers for balance updates

System Architecture



- Frontend provides user interaction.
- Backend validates requests and applies business logic.
- MySQL stores financial, authentication, budget, goal, and reporting data.

Database Design Overview

Database Information

- Database name: `Personal_Finance`
- DBMS: MySQL
- Character set: `utf8mb4`
- Collation: `utf8mb4_0900_ai_ci`

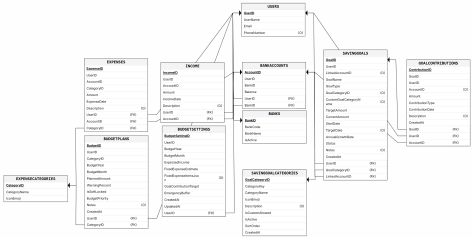
Main Design Groups

- Core financial entities
- Authentication and security entities
- Transaction import and linking entities
- Reporting views and SQL routines

Database Design Goal

The schema is designed to reduce redundancy, preserve financial data integrity, and support reporting through SQL objects.

Core ERD: Overview



Main Focus

- Main financial workflow
- User-owned bank accounts
- Income and expense records
- Monthly budgets and saving goals

Purpose

The Core ERD models the main financial tables used for account tracking, transaction recording, budgeting, goal tracking, and reporting.

Core ERD: Entities and Relationships

Core Entities

- `Users`, `Banks`, `BankAccounts`
- `Income`, `Expenses`, `ExpenseCategories`
- `BudgetSettings`, `BudgetPlans`
- `SavingGoals`, `GoalContributions`

Main Relationships

- One user can own many accounts, transactions, budgets, and goals.
- Each account belongs to one user and one bank.
- Each income or expense is linked to one account.
- Each saving goal can have many contributions.

Rationale

The design keeps financial records separated by user and avoids repeated bank names, category names, and user details.

Core ERD: Design Details

Normalization

- `Banks` is separated from `BankAccounts`.
- `ExpenseCategories` is separated from `Expenses`.
- `BudgetSettings` stores monthly setup.
- `BudgetPlans` stores category-level plans.

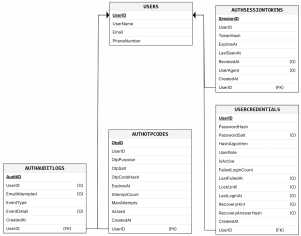
Financial Logic

- Income increases account balance.
- Expenses decrease account balance.
- Goals store long-term targets.
- Contributions record goal progress.

Key Point

The Core ERD provides a normalized foundation for income tracking, expense tracking, budgeting, saving goals, and financial reports.

Authentication ERD: Overview



Security Focus

- Separate user profile from login data.
- Store credentials independently.
- Support OTP and session management.
- Record authentication audit history.

Purpose

The Authentication ERD protects user accounts by separating profile data from credentials, OTP records, session tokens, and audit logs.

Authentication ERD: Entities and Relationships

Security Entities

- `Users`
- `UserCredentials`
- `AuthOtpCodes`
- `AuthSessionTokens`
- `AuthAuditLogs`

Main Relationships

- One user has one credential record.
- One user can request many OTP codes.
- One user can have many login sessions.
- Audit logs link to a user when known.
- Failed or unmatched login events may have nullable `UserID`.

Rationale

Authentication data is separated from financial data to improve security, clarity, and maintainability.

Authentication ERD: Design Details

Stored Security Data

- Password hashes and user roles
- Account status and recovery data
- Hashed OTP codes
- Hashed session tokens

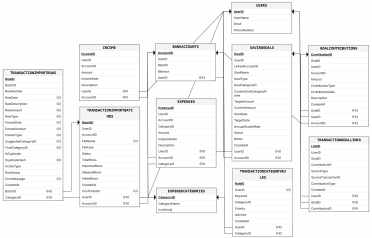
Security Benefits

- Passwords are not stored in plain text.
- OTP and session tokens can expire.
- Sessions can be revoked.
- Audit logs support traceability.

Key Point

This ERD creates a separate security layer for login, account recovery, session validation, and authentication event tracking.

Transaction Import ERD: Overview



Import Focus

- CSV/Excel transaction import
- File-level import batches
- Row-level transaction preview
- Category suggestion and duplicate detection

Purpose

The Transaction Import ERD supports import preview, category suggestion, duplicate detection, confirmation, and transaction-goal traceability.

Transaction Import ERD: Entities and Relationships

Import Entities

- `TransactionImportBatches`
- `TransactionImportRows`
- `TransactionCategoryRules`
- `TransactionGoalLinks`
- `Income`, `Expenses`
- `GoalContributions`

Main Relationships

- One batch contains many import rows.
- One user can create many import batches.
- Category rules map keywords to categories.
- Goal links connect transactions to goal contributions.

Rationale

The design separates uploaded files, preview rows, category rules, confirmed transactions, and goal contribution links.

Transaction Import ERD: Design Details

Import Control

- Batch table stores file metadata.
- Row table stores parsed transaction data.
- Rows can be reviewed before confirmation.
- Duplicate hash helps prevent repeated records.

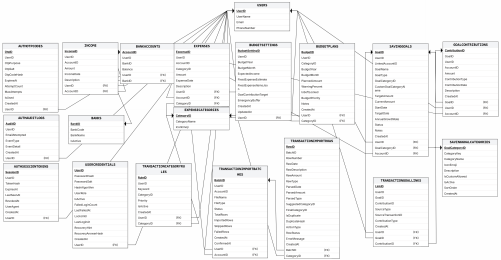
Goal Traceability

- Imported rows may become income or expenses.
- Goal links connect transactions with goals.
- Each link refers to one goal contribution.
- Conditional source reference is checked by backend logic.

Key Point

The import design gives the system control before imported data becomes official financial records.

Full ERD Reference



Combined Schema

- Core financial tables
- Authentication and security tables
- Import staging tables
- Goal linking tables

Use in Report

The full ERD is kept as a reference diagram because it is too dense to explain in one slide.

Normalization

First Normal Form

Each table stores atomic values, and each row is uniquely identified by a primary key.

Second Normal Form

Non-key attributes depend on the whole key. Most tables use surrogate primary keys such as `UserID`, `IncomeID`, and `ExpenseID`.

Third Normal Form

Non-key attributes do not depend on other non-key attributes. Lookup data is separated into independent tables.

Examples:

- `Banks` is separated from `BankAccounts`.
- `ExpenseCategories` is separated from `Expenses`.
- `Users` is separated from `UserCredentials`.

Constraints, Indexing, and Data Integrity

Integrity Mechanisms

- Primary keys
- Foreign keys
- Unique constraints
- Check constraints
- Indexes for common lookups
- Triggers

Examples

- Unique email and phone number
- Positive transaction amount
- Valid budget month and year
- Indexed user, account, budget, and report filters
- Account balance synchronization
- Insufficient balance prevention

Main Result

The database protects financial records from duplication, invalid values, and inconsistent balance updates.

SQL Implementation Overview

SQL File	Purpose
`schema.sql`	Tables, constraints, indexes
`sample_data.sql`	Representative test data
`views.sql`	Reporting views
`procedures.sql`	Stored procedures
`functions.sql`	Financial functions
`triggers.sql`	Automatic balance updates
`security.sql`	Database users and privileges
`migrations/`	Incremental database updates

Implementation Focus

The SQL layer implements data storage, reporting logic, financial calculation, validation, and security.

Views for Reporting: Overview

Why Views Are Used

- Store reusable reporting queries in the database.
- Simplify complex joins and aggregations.
- Keep backend report logic cleaner.
- Support dashboard, budget, balance, and goal reports.

Key Idea

Views act as prepared report layers, allowing the backend to retrieve summarized financial data more easily.

Main Examples

- `vw_budget_vs_actual_monthly`
- `vw_account_balance_history`

View: Budget vs Actual

Purpose

- Compares planned budget with actual spending.
- Shows remaining budget and alert level.

```
SELECT *
FROM vw_budget_vs_actual_monthly
LIMIT 10;
```

BudgetID	UserID	UserName	CategoryID	CategoryName	BudgetYear	BudgetMonth	PlannedAmount	WarningPercent	SpentAmount	RemainingBudget	AlertLevel	AlertSortOrder
24	4	Pham Gia Huy	5	Mua s???m	2026	4	1400000.00	80.00	1500000.00	-100000.00	EXCEEDED	1
19	10	Ngo Tuan Bach	5	Mua s???m	2026	3	2500000.00	80.00	2800000.00	-300000.00	EXCEEDED	1
15	8	Phan Duc Long	3	Gi??o d???c	2026	3	2100000.00	80.00	2200000.00	-100000.00	EXCEEDED	1
11	6	Vu Khanh Nam	6	S???c kh???e	2026	3	1500000.00	80.00	1700000.00	-200000.00	EXCEEDED	1
5	3	Le Thu Ha	6	S???c kh???e	2026	3	1700000.00	80.00	1800000.00	-100000.00	EXCEEDED	1
3	2	Tran Quoc Bao	8	Thu?? nh??	2026	3	5800000.00	80.00	6000000.00	-200000.00	EXCEEDED	1
30	10	Ngo Tuan Bach	5	Mua s???m	2026	4	3200000.00	80.00	2800000.00	400000.00	WARNING	2
29	9	Dang Bao Chau	6	S???c kh???e	2026	4	1000000.00	80.00	900000.00	100000.00	WARNING	2
28	8	Phan Duc Long	3	Gi??o d???c	2026	4	2300000.00	80.00	2200000.00	100000.00	WARNING	2
26	6	Vu Khanh Nam	6	S???c kh???e	2026	4	600000.00	80.00	500000.00	100000.00	WARNING	2

MySQL result of `vw_budget_vs_actual_monthly`

View: Account Balance History

Purpose

- Shows income and expense transactions over time.
- Calculates running balance for each account.

```
SELECT *
FROM vw_account_balance_history
LIMIT 10;
```

UserID	UserName	AccountID	BankName	TransactionDate	TransactionType	SortOrder	ReferenceID	AmountSigned	RunningBalance
1	Nguyen Minh Anh	1	Vietcombank	2025-01-01 08:00:00	INCOME	1	16	20000000.00	20000000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-01-03 09:00:00	EXPENSE	2	16	-3500000.00	16500000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-01-10 19:00:00	EXPENSE	2	271	-2200000.00	14300000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-01-18 08:30:00	EXPENSE	2	526	-1000000.00	13300000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-02-01 08:00:00	INCOME	1	15	20000000.00	33300000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-02-03 09:00:00	EXPENSE	2	15	-3500000.00	29800000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-02-10 19:00:00	EXPENSE	2	270	-2200000.00	27600000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-02-18 08:30:00	EXPENSE	2	525	-1000000.00	26600000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-03-01 08:00:00	INCOME	1	14	20000000.00	46600000.00
1	Nguyen Minh Anh	1	Vietcombank	2025-03-03 09:00:00	EXPENSE	2	14	-3500000.00	43100000.00

MySQL result of `vw_account_balance_history`

Stored Procedure: Monthly Closure Summary

Purpose

- Centralizes monthly financial summary logic.
- Returns total income, total expense, and net saving.

```
CALL sp_monthly_closure_summary(2025, 4);
```

	YearMonth	MonthlyIncome	MonthlyExpense	NetSaving
▶	2025-04	156000000.00	85700000.00	70300000.00

MySQL result of `sp_monthly_closure_summary`

User-defined Functions: Financial Summary

Purpose

- Provides reusable financial calculations.
- Calculates total income, total expense, and net saving for a user.

```sql
SELECT
    fn_total_income_by_user(1) AS TotalIncome,
    fn_total_expense_by_user(1) AS TotalExpense,
    fn_net_saving_by_user(1) AS NetSaving;
```

	TotalIncome	TotalExpense	NetSaving
▶	348000000.00	115200000.00	232800000.00

MySQL result of user-defined financial functions

Trigger: Database-Level Consistency

Key Highlight

Triggers make the database actively protect financial consistency, not only store data passively.

How Triggers Maintain Balance

- Income insert/update/delete automatically adjusts account balance.
- Expense insert/update/delete automatically adjusts account balance.
- Expense validation prevents spending more than available balance.
- Balance remains consistent even if data changes are executed directly in MySQL.

```sql
INSERT INTO Income
(UserID, AccountID, Amount, IncomeDate, Description)
VALUES (1, 1, 123456.00, NOW(), 'Trigger test income');
```

Main Message

This is the strongest database feature because it protects financial data integrity at the database layer.

Trigger Verification: Balance Consistency

Before transaction → Insert income → Balance updated automatically

AccountID	UserID	BalanceBefore
1	1	212800000.00
NULL	NULL	NULL

Step 1: Balance before insert

15 09:08:53 INSERT INTO Income (UserID, AccountID, Amount, IncomeDate, Description) VALUES (1, 1, 1... 1 row(s) affected

Step 2: Test income inserted

AccountID	UserID	BalanceAfter
1	1	212923456.00
NULL	NULL	NULL

Step 3: Balance after insert

Verification Result

The account balance increased automatically after the income record was inserted, proving that the trigger keeps financial data consistent.

Database Security

Database Users

- `pf_app`: backend runtime user
- `pf_readonly`: read-only user

Principle

- Avoid using MySQL root account for runtime
- Grant only required privileges

Grants for pf_app@%:

```
GRANT SELECT, INSERT, UPDATE, DELETE, EXECUTE, SHOW VIEW ON `personal_finance`.* TO `pf_app`@`%`
```

MySQL grants for `pf_app`

Security and Deployment: Bootstrap Admin

Why Bootstrap Is Needed

- Creates the first admin account after database reset.
- Enables initial login and admin user management.
- Supports repeatable setup for Docker Compose and Railway.

Security Benefit

- Admin credentials are read from environment variables.
- Passwords are not hard-coded in `sample_data.sql`.
- Authentication secrets are separated from sample financial data.

Key Point

The bootstrap script safely initializes the first admin account while keeping credentials outside SQL seed files.

Web Review

Frontend workflow and application demonstration

https://vaulted.up.railway.app/login

Backend and Frontend Summary

Backend

- FastAPI REST API
- Router, service, repository layers
- Authentication and validation
- Connects frontend with MySQL

Frontend

- Next.js and TypeScript
- Tailwind CSS interface
- Dashboard, transactions, budgets
- Goals, reports, imports, admin page

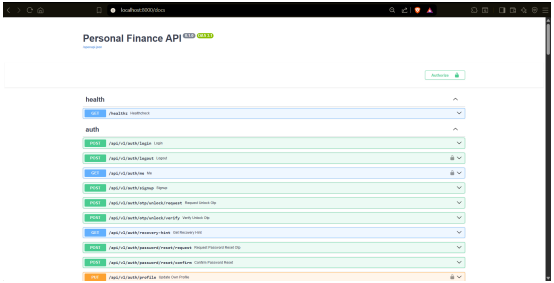
Note

The detailed web workflow is demonstrated in the project video.

Testing and Deployment

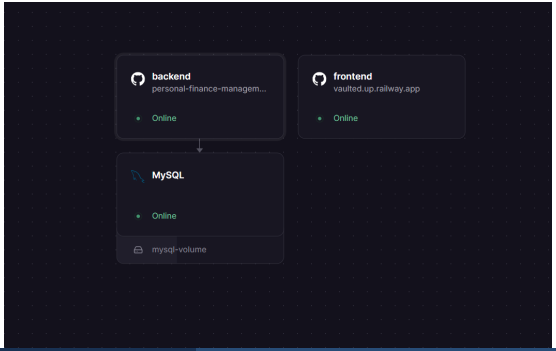
Testing Evidence

- MySQL query results
- FastAPI Swagger UI
- Backend health check
- Dashboard after login



Deployment

- Local: Docker Compose
- Cloud: Railway
- Services: frontend, backend, MySQL
- Configured by environment variables



Conclusion

Main Achievement

The project successfully implements a complete MySQL-backed personal finance management system.

Key Results

- Normalized relational database design
- SQL implementation with advanced database objects
- Reporting views, procedures, functions, and triggers
- Backend and frontend integration
- Local and cloud deployment verification

Future Improvements

- Real bank API integration
- Recurring transactions
- AI spending prediction
- Mobile application

Thank You

m

For Listening

Ngo Tuan Bach

Personal Finance Management System